

Studi'OS

Roadmap - Roadmaps structurées, initialisation de projet et hydratation des tâches

Chantier conçu pour être exécuté par plusieurs sessions IA en parallèle, tout en conservant Studi'OS agnostique au modèle, au fournisseur et au harness.

But produit

Permettre à un humain de définir un projet avec son IA, puis à cette IA de créer dans Studi'OS le projet, sa configuration IA, une roadmap structurée et les tâches associées. La roadmap devient ensuite une source de contexte exploitable par le Dashboard, l'API, le MCP et studio_prepare_context.

Principe	Décision
Source de vérité	Roadmap structurée dans Studi'OS ; le PDF n'est qu'un export.
IA serveur	Aucune. L'IA externe propose ; Studi'OS valide, stocke et expose.
Agnosticisme	Aucune dépendance à Claude, OpenAI, OpenCode, Qwen ou autre fournisseur.
Relation aux tâches	Roadmap = plan ; Tasks = unités de travail ; liens explicites entre les deux.
Validation	Une proposition IA peut rester en brouillon jusqu'à validation humaine.

1. Résultat attendu

À la fin du chantier, un agent externe doit pouvoir partir d'une discussion déjà alignée avec l'utilisateur et effectuer une initialisation contrôlée de projet. Le même résultat doit rester réalisable manuellement depuis le Dashboard ou via l'API.

- R1 - Créer un projet Studi'OS et enregistrer les informations essentielles du projet.
- R2 - Créer une roadmap versionnée/traçable composée de phases, étapes et dépendances.
- R3 - Associer chaque étape à zéro, une ou plusieurs Tasks existantes ou nouvelles.
- R4 - Hydrater les Tasks à partir d'une roadmap sans dupliquer silencieusement le travail.
- R5 - Configurer les ressources IA génériques nécessaires au projet via la Library et les bindings existants.
- R6 - Exposer l'étape courante, les tâches disponibles/bloquées et le contexte pertinent via `studio_prepare_context`.
- R7 - Afficher une vue Roadmap simple dans le Dashboard, synchronisée avec les Tasks.
- R8 - Permettre une proposition IA en brouillon puis une validation/modification humaine avant activation.
- R9 - Conserver provenance, audit, permissions, idempotence et compatibilité avec l'architecture actuelle.
- R10 - Permettre l'export PDF d'une roadmap sans faire du PDF la source de vérité.

Invariants non négociables

- Aucun LLM, agent ou raisonnement hébergé par le serveur Studi'OS.
- Le domaine Roadmap ne contient aucune notion propre à un fournisseur ou à un harness.
- L'API HTTP reste la surface canonique ; le MCP expose un sous-ensemble adapté aux agents.
- Une Task peut exister sans Roadmap. La Roadmap enrichit l'organisation, elle ne remplace pas le domaine Tasks.
- Les opérations générées par une IA portent une provenance explicite et respectent les mêmes permissions que les autres écritures.
- Les créations en lot doivent être idempotentes ou disposer d'une stratégie de reprise sûre.
- Le Dashboard doit rester simple : détails techniques relégués à la divulgation progressive / Inspector.

2. Stratégie de parallélisation

Le chantier doit privilégier des branches/worktrees indépendants et des contrats stabilisés tôt. P0 et P1 servent de gate commun. Après P1, plusieurs lanes peuvent progresser simultanément. Une session d'intégration reste seule responsable des réconciliations.

Lane	Travail	Peut démarrer
A - Domain/API	P2 + P3	Après P1
B - MCP/Context	P4 + P6	P4 après contrats P1/P3 ; P6 après surfaces minimales
C - Initialization	P5	Après contrats P1 ; mock API autorisé
D - Dashboard	P7	Après contrats UI/API figés ; fixtures autorisées
E - Export/Docs	P9	Après schéma Roadmap stabilisé
Integration	P8 + P10	Après convergence A/B/C/D

Règle Git : chaque session travaille sur une branche dédiée depuis une baseline commune. Aucun cherry-pick/merge opportuniste entre lanes. Les contrats partagés sont modifiés uniquement par la lane propriétaire ou via une réconciliation explicite.

P0 - Architecture Gate & audit de l'existant

Objectif : Figer le périmètre et empêcher la feature Roadmap de dupliquer des concepts déjà présents.

Dépendances : Aucune.

Parallélisation : Non. Une seule session d'architecture.

P0.1 - Cartographier les modèles Project, Task, Library, Runtime Registry, bindings, Review, audit, événements, API et MCP existants.

P0.2 - Identifier les invariants de scope/ownership, idempotence, permissions, provenance et suppression déjà utilisés.

P0.3 - Décider si Roadmap est un nouveau domaine ou une extension d'un domaine existant ; documenter la décision.

P0.4 - Définir la frontière Roadmap / Task : statut, progression, dépendances, ordre et source de vérité.

P0.5 - Définir le cycle de vie : draft/proposed/active/completed/archived, avec transitions autorisées.

P0.6 - Définir la stratégie de versionnement et de modification d'une roadmap active.

P0.7 - Définir le modèle de provenance des propositions IA et modifications humaines.

P0.8 - Produire l'ADR/DEC et la matrice REQUIREMENT -> DOMAIN -> API -> MCP -> UI -> TEST.

Gate P0 : aucun code produit avant validation de l'architecture.

P1 - Contrats communs et schéma Roadmap

Objectif : Fournir une base contractuelle stable permettant aux autres sessions de travailler en parallèle.

Dépendances : P0 accepté.

Parallélisation : Faible. Une session propriétaire des contrats.

P1.1 - Définir Roadmap, RoadmapPhase/RoadmapStep (ou structure retenue par P0), statuts et identifiants.

P1.2 - Définir ordre stable, dépendances entre étapes et règles anti-cycle.

P1.3 - Définir les liens Step <-> Task sans rendre task.roadmap_id obligatoire.

P1.4 - Définir les champs : objectif, contexte, instructions, critères d'acceptation, notes et métadonnées bornées.

P1.5 - Définir les contrats de création en lot utilisés pour l'hydratation.

P1.6 - Définir un format d'import/export JSON neutre et versionné.

P1.7 - Définir les erreurs structurées : conflit, cycle, référence absente, état invalide, duplicate/idempotency.

P1.8 - Ajouter tests de contrats et fixtures partagées pour les lanes parallèles.

Gate P1 : contrats considérés gelés pour les lanes A-D. Toute rupture ultérieure passe par réconciliation.

P2 - Domaine Roadmap & persistance

Objectif : Implémenter la source de vérité serveur.

Dépendances : P1.

Parallélisation : Lane A ; peut avancer pendant P4/P5/P7.

P2.1 - Créer les modèles de persistance et migrations réversibles.

P2.2 - Implémenter CRUD métier avec scopes/ownership existants.

P2.3 - Implémenter ordre des phases/étapes et réordonnement atomique.

P2.4 - Implémenter dépendances et détection de cycles.

P2.5 - Implémenter activation/archivage et transitions de statut.

P2.6 - Implémenter liens Step <-> Task et calcul de progression dérivé.

P2.7 - Implémenter création en lot / hydratation idempotente.

P2.8 - Enregistrer provenance et audit selon les mécanismes existants.

P2.9 - Tests unitaires : permissions, conflits, cycles, rollback, idempotence, progression.

P3 - API HTTP canonique

Objectif : Exposer toute la capacité Roadmap indépendamment de l'IA.

Dépendances : P2 et contrats P1.

Parallélisation : Lane A ; sous-session possible pour tests OpenAPI après endpoints.

P3.1 - Endpoints liste/création/détail/modification/archivage Roadmap.

P3.2 - Endpoints phases/steps, réordonnancement et dépendances.

P3.3 - Endpoints de liaison/déliaison aux Tasks.

P3.4 - Endpoint de prévisualisation d'hydratation : ce qui sera créé/modifié sans écrire.

P3.5 - Endpoint d'application de l'hydratation avec clé d'idempotence.

P3.6 - Endpoints import/export du format neutre.

P3.7 - Permissions, audit, erreurs cohérentes et OpenAPI.

P3.8 - Tests API complets et non-régression des routes existantes.

P4 - Surface MCP Roadmap

Objectif : Donner aux agents une surface compacte sans répliquer toute l'API.

Dépendances : P1 ; idéalement P3 figé.

Parallélisation : Lane B en parallèle de P2/P3/P5/P7.

P4.1 - Définir le sous-ensemble MCP minimal selon la règle HTTP canonique / MCP adapté.

P4.2 - Ajouter un outil de lecture synthétique de roadmap et de position courante.

P4.3 - Ajouter un outil de proposition/création de roadmap structurée.

P4.4 - Ajouter preview puis apply pour l'hydratation des Tasks.

P4.5 - Ajouter les opérations bornées nécessaires aux mises à jour d'étapes.

P4.6 - Garantir budgets de sortie, pagination/limites et erreurs structurées.

P4.7 - Tests MCP et parité sémantique avec l'API.

Éviter un outil MCP par endpoint. Les outils doivent correspondre à des intentions d'agent : lire le plan, proposer un plan, prévisualiser l'hydratation, appliquer l'hydratation, mettre à jour l'avancement.

P5 - Project Initialization

Objectif : Permettre à une IA externe de transformer un projet aligné avec l'humain en workspace Studi'OS cohérent.

Dépendances : P1 ; API Roadmap peut être mockée jusqu'à P3.

Parallélisation : Lane C indépendante.

P5.1 - Définir un ProjectInitializationPlan neutre : project, roadmap, task plan, resources, bindings.

P5.2 - Ajouter une opération de preview sans effet de bord.

P5.3 - Valider références Library, compatibilités et droits avant toute écriture.

P5.4 - Appliquer l'initialisation de façon transactionnelle ou avec reprise sûre documentée.

P5.5 - Réutiliser les mécanismes existants de création Project/Task/Library/Binding au lieu de les dupliquer.

P5.6 - Permettre un mode draft/proposed : rien n'est activé avant validation humaine lorsque demandé.

P5.7 - Retourner un résumé : objets créés, réutilisés, ignorés, conflits et provenance.

P5.8 - Tester replays, erreurs partielles, ressources absentes et initialisation sans aucune IA.

Le serveur ne doit jamais décider de la roadmap. L'agent externe fournit un plan structuré ; Studi'OS ne fait que le valider et l'appliquer.

P6 - Intégration à studio_prepare_context

Objectif : Faire de la roadmap une source de contexte utile sans gonfler le contexte agent.

Dépendances : P4 + modèle Roadmap fonctionnel.

Parallélisation : Lane B ; peut être sous-session distincte de P4.

P6.1 - Ajouter un résumé roadmap borné au Context Package.

P6.2 - Exposer étape/phase courante et prochaine unité de travail disponible.

P6.3 - Inclure dépendances bloquantes et critères d'acceptation pertinents.

P6.4 - Inclure uniquement les Tasks et ressources IA pertinentes à l'étape courante.

P6.5 - Respecter le budget Context Package et omitted[] existant.

P6.6 - Définir le comportement quand aucune roadmap n'existe ou qu'elle est en draft.

P6.7 - Tests de sélection, budgets, déterminisme et fallback serveur indisponible.

P7 - Dashboard Roadmap

Objectif : Offrir une lecture humaine immédiate et une validation simple des propositions IA.

Dépendances : Contrats P1 ; intégration réelle après P3.

Parallélisation : Lane D en parallèle, avec fixtures.

P7.1 - Ajouter l'onglet Roadmap au workspace Projet sans casser la navigation actuelle.

P7.2 - Vue verticale des phases avec progression, état et étape actuelle.

P7.3 - Détail d'une étape : objectif, tâches, critères, dépendances et ressources IA.

P7.4 - Vue Plan / Exécution : plan complet vs travail disponible maintenant.

P7.5 - Afficher tâches disponibles, en cours et bloquées sans créer un second Kanban.

P7.6 - Synchroniser visuellement progression Roadmap avec les Tasks.

P7.7 - Écran de proposition : résumé, diff, modifier, approuver ou rejeter.

P7.8 - Création/édition manuelle complète afin que la feature fonctionne sans IA.

P7.9 - Progressive disclosure : IDs, provenance, bindings et détails techniques dans Inspector/détails avancés.

P7.10 - Responsive, accessibilité, états vide/loading/error et tests frontend/E2E.

P8 - Review, validation humaine & événements

Objectif : Sécuriser le passage d'une proposition IA vers le plan officiel.

Dépendances : P2/P3/P5/P7.

Parallélisation : Session d'intégration ; éviter développement parallèle sur les mêmes contrats.

P8.1 - Réutiliser la Review Queue existante si elle correspond au besoin ; sinon justifier une extension minimale.

P8.2 - Créer le flux proposé -> approved/changes_requested/rejected -> active.

P8.3 - Afficher un diff compréhensible avant approbation : phases, tâches, dépendances, ressources/bindings.

P8.4 - Émettre les événements nécessaires pour Dashboard/consommateurs sans coupler la roadmap au Git Watcher.

P8.5 - Tracer auteur/provenance, date, décision et version approuvée.

P8.6 - Tester concurrence : humain modifie pendant qu'une proposition IA est en attente.

P9 - Import, export PDF et portabilité

Objectif : Conserver les roadmaps partageables sans enfermer les données dans un document.

Dépendances : P1 stabilisé ; export final après P7.

Parallélisation : Lane E largement indépendante.

P9.1 - Finaliser export/import JSON neutre avec version de schéma.

P9.2 - Créer un export PDF lisible inspiré des roadmaps de développement utilisées pour Studi'OS.

P9.3 - Inclure phases, tâches, dépendances, critères et statut, sans secrets ni détails privés par défaut.

P9.4 - Permettre export depuis Dashboard/API.

P9.5 - Tester round-trip JSON et génération PDF sur petites/grandes roadmaps.

P10 - Réconciliation, E2E et clôture

Objectif : Assembler les lanes, vérifier les invariants et tester le produit dans un scénario réel.

Dépendances : P2 à P9.

Parallélisation : Une session intégratrice + sessions de tests non-modifiantes possibles.

P10.1 - Rebaser/réconcilier les lanes sur une baseline unique ; résoudre explicitement les divergences de contrats.

P10.2 - Exécuter migrations, tests backend, contrats, MCP, dashboard, Playwright et non-régression globale.

P10.3 - Scénario E2E : discussion déjà alignée -> initialization preview -> validation -> création Project -> Roadmap -> Tasks -> Library/Bindings.

P10.4 - Ouvrir une nouvelle session avec un autre harness/modèle et vérifier qu'elle reprend le projet via studio_prepare_context.

P10.5 - Modifier une Task depuis le Dashboard et vérifier la progression Roadmap.

P10.6 - Modifier la Roadmap manuellement et vérifier API/MCP/context.

P10.7 - Tester projet sans IA, projet sans roadmap, roadmap draft, roadmap archivée et reprise après erreur.

P10.8 - Audit final d'agnosticisme : aucune dépendance fournisseur/model/harness dans le domaine ou les contrats génériques.

P10.9 - Mettre à jour documentation utilisateur simple + documentation technique/DEC.

3. Plan de sessions recommandé

Pour réduire le temps total, lancer les sessions suivantes une fois P1 fusionné sur la branche de référence :

Session	Branche suggérée	Phases	Règle
S-A	feature/roadmap-domain-api	P2-P3	Propriétaire DB/API
S-B	feature/roadmap-mcp-context	P4 + P6	Ne modifie pas le domaine sans accord
S-C	feature/project-initialization	P5	Réutilise les services existants
S-D	feature/roadmap-dashboard	P7	Fixtures jusqu'à API stable
S-E	feature/roadmap-export	P9	JSON/PDF uniquement
S-I	feature/roadmap-integration	P8 + P10	Réconciliation finale

Ordre conseillé

Session 0 : P0. Validation humaine. Session 1 : P1 puis fusion. Vague parallèle : S-A + S-B + S-C + S-D + S-E. Convergence : S-I réalise P8, réconciliation puis P10.

Critères de clôture produit

- ✓ - Une roadmap peut être créée et gérée entièrement sans IA.
- ✓ - Une IA externe peut proposer une initialisation complète sans dépendance à son fournisseur.
- ✓ - Le preview permet de comprendre précisément ce qui sera écrit avant application.
- ✓ - L'hydratation est sûre en cas de replay et ne duplique pas les Tasks.
- ✓ - La progression Roadmap est dérivée des Tasks liées et reste cohérente entre API, MCP et Dashboard.
- ✓ - `studio_prepare_context` expose le travail courant sans injecter la roadmap complète dans le contexte.
- ✓ - Un second agent utilisant un autre harness/modèle peut reprendre le projet sans connaissance de la session créatrice.
- ✓ - Toutes les opérations sensibles respectent permissions, audit et provenance.
- ✓ - Les suites de tests existantes restent vertes.

Test dogfood recommandé : utiliser cette roadmap elle-même comme premier jeu de données. L'agent doit créer le projet/chantier associé dans Studi'OS, importer les phases P0-P10, hydrater les tâches, établir leurs dépendances puis travailler à partir de `studio_prepare_context`. Cela teste immédiatement la feature de coordination avec un cas réel.